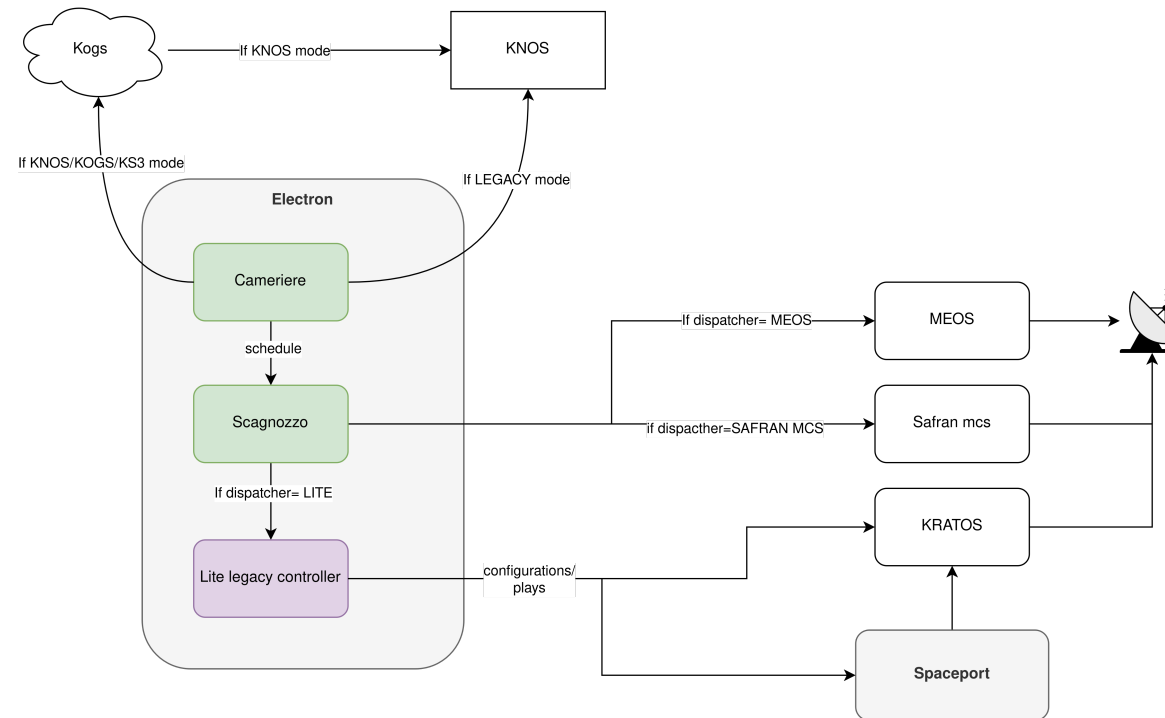


Extension scheduler_cameriere_scagnozzo

This extension deploys Scagnozzo and Cameriere to electron. You can use either KNOS, KS3, LEGACY or KOGS to schedule either MEOS, Legacy_controller, Safran mcs.



Extension scheduler_cam...

Cameriere (system sched...

Scagnozzo (dispatcher)

Deployment

Secrets in vault

General variables

DISPATCHER: MEOS mode

DISPATCHER: SDA mode

DISPATCHER: LITE mode

Cameriere (system schedule handler)

TLDR: Cameriere communicated with KOGS or KNOS Legacy API using a few different protocols. Which systems Cameriere get's information for is controlled using KSAT IDs(API key). This information is cached for the site/system by Cameriere, and scagnozzo uses this information to know when it shall execute tasks(send schedule to meos). This module is configured using variables in the deployment inventory.

In KSAT contact information is stored in the central databases in many different varieties and is emitted to the site in numerous different manners. This scheduler aims to unify all the different formats, providing a single

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camiere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

interface which the Electron Control System and other components can rely on.

The scheduler supports fetching information from the following sources:

KS3 (Default): The ks3-based scheduler relies on the ks3 endpoints for fetching contact information.

This is the desired scheduler to use in the long-term, and should be deployed whenever possible. In order to make use of this scheduler mode, all relevant contact information must be handled by Kogs.

KOGS: The kogs-based scheduler relies on Kogs' internal endpoints, enabling us to fetch contact data from Kogs. This enables us to schedule contacts which are located on migrated antennas only. This mode serves as a temporary migration mode, allowing us to roll out this scheduler in the first place on migrated antennas, which are already controller by another existing Ks3-based scheduler. **This mode exists only for migration purposes, and should normally not be used.**

KNOS: The knos-based scheduler relies on Kogs' integration endpoints, enabling us to fetch contact data from Knos. This mode enables us to start scheduling contacts that Knos would normally schedule. This paves the path forwards to migrating additional antennas over to Kogs. Note that this scheduler requires all assets to be set up in Kogs before it is rolled out! Its primary use-case is to enable us to be agnostic to the source of truth for contact information. It enables us to migrate away from Knos-based contact data to Kogs-based contact data. It should not normally be used.

LEGACY: The legacy-based scheduler bypasses Kogs entirely, and goes directly to Legacy instead. This mode enables us to start taking scheduling responsibility from Knos, bringing us a little bit more flexibility. This mode is intended to be used on antenna systems where we have no administrative responsibility moved to Kogs. Note that all assets such as spacecrafts and systems must be set up in Kogni, and properly linked to the Knos assets, to use this scheduler mode. This mode can be used for contacts which are handled by Knos.

The scheduler caches all information locally on-site according to its configured retention scheme. It can either store all of its information in memory, on the file system, or in an external database, depending on the needs of the antenna system.

The scheduler offer an [API surface](#) for the Electron Control System and other clients to interact with, allowing for the various clients to gain insight into schedule information. This API approach enable additional clients to access the information without having to make changes to the scheduler itself when new clients are introduced. Clients also do not need to expose an API surface themselves, as they are responsible for initiating the communication.

Refer to full [README](#) for detailed information and configuration instructions.

Scagnozzo (dispatcher)

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

TLDR: Monitors schedule information cached by Cameriere. It has the following modes available: LITE, MEOS, MAX, SDA, SAFRAN_MCS

Executes tasks on command. For MEOS Control, it will deliver a schedule xml using meos mcmw(on port 10000), or filedrop. It can also deliver TLE to MEOS Control systems. This module is configured using variables in the deployment inventory.

Contacts must be distributed to a variety of sub-systems prior to spacecraft communication. Traditionally, all components in the chain have been scheduled ahead-of-time from a central dispatcher, KNOS. All contact state has been residing in all components, which has led to a variety of state inconsistencies in the past.

Scagnozzo represents a new architecture where sub-systems are scheduled just-in-time, to avoid state inconsistencies when contacts change. Rather than solving the issue of ensuring all state is kept up-to-date on all sub-systems, the data is only distributed on-demand.

This component reacts a few minutes before a contact is due to start, and ensures that all sub-systems are ready when the contact starts. Scagnozzo uses a combination of information from the contact itself, as well as static configuration in its startup phase, to determine which sub-systems to schedule and how to do so.

Refer to full [README](#) for detailed information and configuration instructions.

i

As of June 6th 2025, Scagnozzo in LITE mode does not properly sign the contacts after it has finished. Until then, use scheduler_brokkr to schedule legacy controller.

Deployment

The deployment of Cameriere and Scagnozzo is defined in [GSC Ansible Awx](#) in the extension `templates/deploy_service/electron/extensions` to deploy both containers to the electron VM, and in `templates/deploy_service/spaceport/extensions` to deploy Scagnozzo to Spaceport. **This deployment describes the Electron extension.**

The description below describes the general variables to set in your electron inventory as (host) variables, + each individual scenario wheather dispatcher is MEOS, LITE, MAX, SASSY, SDA or a combination of them.

Secrets in vault

SECRET NAME	DESCRIPTION
ks3_api_key	KSAT1-PLAIN <api-key> from Kogni. Guide

SECRET NAME

DESCRIPTION

cameriere_client_api_key

KSAT1-PLAIN <api-key> (as of now, make one yourself)

General variables

Specify in inventory (host) vars if necessary

VARIABLE	DEFAULT VALUE	ALLOWED VALUES	COMMENT
cameriere_debug	false	true/false	Debug mode for logging Cameriere
cameriere_scheduler	'KS3'	See env var SCHEDULER	
cameriere_signer	'KS3'	See env var SIGNER	
cameriere_commit_ahead_seconds	'300'	integer	seconds
spaceport_host			
application_mode	'ELECTRON'	'ELECTRON', 'SPACEPORT'	
scheduler_dispatcher	'NONE'	'MEOS,LITE,MAX,SDA,SAFRAN_MCS,SASSY'	See docs

DISPATCHER: MEOS mode

VARIABLE	VALUES	COMMENT
scheduler_dispatcher	'MEOS'	
scheduler_dropbox	'SSH'	Other options possible. See docs
scheduler_meos_host	ip address of meos vm	
scheduler_meos_port	integer	Defaults to '10000'
scheduler_meos_username	string	Defaults to 'meos'
scheduler_meos_password	string (can add in vault)	Defaults to default password
scheduler_ssh_dropbox_host	ip:port	i.e 127.0.0.1:22

DISPATCHER: SDA mode

VARIABLE	VALUES	COMMENT
scheduler_dispatcher	'SDA'	
scheduler_dropbox	'SSH'	Other options possible. See docs
scheduler_meos_host	ip address of meos vm	
scheduler_meos_port	integer	Defaults to '10000'
scheduler_meos_username	string	Defaults to 'meos'
scheduler_meos_password	string (can add in vault)	Defaults to default password
scheduler_ssh_dropbox_host	ip:port	i.e 127.0.0.1:22

DISPATCHER: LITE mode

VARIABLE	VALUES	COMMENT
scheduler_dispatcher	'LITE'	